

Periodic Net View Specification

Signature Projection, Source Binding, and Periodic-Dimensionality Diagnostics

mdstats

2026-07-18

Contents

1 Purpose	2
2 Motive	2
3 Algorithmic provenance	2
4 Module boundary	3
5 Public API	3
6 Signature policy	3
6.1 Vertex signature fields	3
6.2 Edge signature fields	4
6.3 NetViewPolicy	4
6.3.1 Input constraints	4
6.3.2 Serialization	4
7 Component diagnostics	4
7.1 PeriodicNetComponent	4
8 PeriodicNetView	5
8.1 Meaning	5
8.2 Convenience properties	5
8.3 Source lookup	6
8.4 Serialization	6
9 Builder	6
9.1 Input constraints	6
9.2 Output guarantees	6
10 Periodic translation analysis	6
10.1 Quotient components	6
10.2 Fundamental cycle gains	7
10.3 Rank	7
10.4 Finite index and lifted connectedness	7
11 Natural-tiling precondition	7
12 Digest and equality semantics	8
13 Edge cases and warnings	8
13.1 Ignoring a field does not merge records	8

13.2 Structural filtering is out of scope	8
13.3 Full rank does not imply connected lift	8
13.4 Supercell and hidden-translation interpretation	8
13.5 Disconnected and lower-dimensional structures	8
13.6 Policy extension	8
14 Focused test requirements	9
15 Non-goals	9
16 References	9

1 Purpose

`periodic_net_view.py` defines the exact periodic multigraph interpretation under which symmetry and natural tiling will be computed.

The first backend is deliberately a **signature projection** of one immutable `FrameworkTopology`:

$$V_{\text{view}} = V_{\text{framework}}, \quad E_{\text{view}} = E_{\text{framework}}.$$

It changes only the deterministic vertex and edge signatures that a later automorphism must preserve. It does not remove, contract, merge, or reconstruct any graph record.

Runtime/API version:

`mdstats 0.19.14a0`

2 Motive

`FrameworkTopology` records what the framework connectivity physically is. The same graph can be interpreted under different equivalence policies:

- an unlabeled framework T -net may allow Si and Al vertices to exchange;
- a chemically decorated view may require Si and Al to remain distinct;
- selected linker or edge decorations may be preserved or ignored.

The automorphism group therefore depends on both the graph and the declared signature policy:

$$\text{Aut}(G, \sigma_V, \sigma_E).$$

A dedicated immutable view prevents symmetry policy from mutating or overloading the source topology object. It also gives downstream symmetry and tiling results one explicit provenance digest.

3 Algorithmic provenance

The periodic quotient-edge representation and closed-walk gain construction follow the vector/quotient-graph treatment of Chung, Hahn, and Klee [1] and Klee [2]. The policy-bound exact automorphism viewpoint follows Delgado-Friedrichs and O’Keeffe [3]. The finite index of a full-rank integer translation subgroup is computed from the greatest common divisor of its maximal minors, equivalently the last determinant divisor in Smith normal form [4].

The following are `mdstats` design decisions:

- a signature-only first backend that preserves exact source orbit sets;
- field-enumerated, serialization-safe built-in policies rather than arbitrary callables;
- descriptive policy labels excluded from scientific digest identity;

- simultaneous storage of source graph and source topology digests;
- explicit per-component cycle-gain, rank, and finite-index diagnostics; and
- the strengthened three-periodic eligibility condition requiring translation subgroup index one.

References [1–4] are repeated in the implementation module comments adjacent to the adopted periodic-gain and lattice-index machinery.

4 Module boundary

```

FrameworkTopology
    |
    | exact immutable vertices, edges, shifts, decorations
    v
NetViewPolicy
    |
    | retained signature fields only
    v
PeriodicNetView
    |
    +--> future periodic-net symmetry
    +--> future PeriodicNetEmbedding
    +--> future natural tiling

```

`PeriodicNetView` is not a second topology catalog. It stores source identifiers, derived signatures, and periodic-net diagnostics while retaining exact mappings to the source `FrameworkTopology` records.

5 Public API

The module is exported through `mdstats.analysis`, not the package root.

```

from mdstats.analysis import (
    CANONICAL_NET_VIEW_POLICY_SCHEMA,
    CANONICAL_PERIODIC_NET_VIEW_SCHEMA,
    PERIODIC_NET_VIEW_DIGEST_ALGORITHM,
    VertexSignatureField,
    EdgeSignatureField,
    NetViewPolicy,
    PeriodicNetComponent,
    PeriodicNetView,
    PeriodicNetViewError,
    PeriodicNetViewInputError,
    PeriodicNetViewSerializationError,
    build_periodic_net_view,
)

```

6 Signature policy

6.1 Vertex signature fields

```

class VertexSignatureField(str, Enum):
    ATOMIC_NUMBER = "atomic_number"

```

The first backend supports only source attributes with deterministic canonical serialization. Omitting `ATOMIC_NUMBER` makes all retained framework vertices share the base signature

```

("framework_vertex",)

```

and therefore permits chemically different vertices to be exchanged if graph incidence also allows it.

Including `ATOMIC_NUMBER` produces signatures of the form

```
("framework_vertex", "atomic_number", 14)
```

6.2 Edge signature fields

```
class EdgeSignatureField(str, Enum):
    EDGE_KIND = "edge_kind"
    RULE_ID = "rule_id"
    LINKER_ATOMIC_NUMBERS = "linker_atomic_numbers"
    LINKER_COUNT = "linker_count"
```

Omitting all fields gives every source edge orbit the base signature

```
("framework_edge",)
```

This permits automorphisms to exchange differently decorated edges, but it never merges them. Parallel source edge records remain separate graph edges.

6.3 NetViewPolicy

```
@dataclass(frozen=True, slots=True)
class NetViewPolicy:
    vertex_fields: tuple[VertexSignatureField, ...] = ()
    edge_fields: tuple[EdgeSignatureField, ...] = ()
    label: str = "unlabeled framework net"
    canonical_schema_version: str = CANONICAL_NET_VIEW_POLICY_SCHEMA
    digest_algorithm: str = PERIODIC_NET_VIEW_DIGEST_ALGORITHM
    digest: str = ""
```

Built-in constructors:

```
NetViewPolicy.unlabeled_framework_net(...)
NetViewPolicy.chemically_decorated(...)
```

6.3.1 Input constraints

- field tuples must contain supported enum values only;
- duplicate fields are rejected;
- fields are sorted canonically before digesting;
- label must be nonempty;
- label is descriptive provenance and is excluded from the semantic policy digest;
- supplied schema, algorithm, and digest values must validate exactly.

6.3.2 Serialization

```
policy.to_dict() -> dict[str, Any]
NetViewPolicy.from_dict(payload) -> NetViewPolicy
```

The serialized form contains the descriptive label, while scientific identity is set by the retained field lists and schema version.

7 Component diagnostics

7.1 PeriodicNetComponent

```
@dataclass(frozen=True, slots=True)
class PeriodicNetComponent:
    component_id: int
```

```

vertex_positions: tuple[int, ...]
edge_positions: tuple[int, ...]
cycle_gain_generators: tuple[tuple[int, int, int], ...]
translation_rank: int
translation_index: int | None

```

`vertex_positions` and `edge_positions` are positions in the view's source-aligned arrays. They are not atom indices or dense identifiers that may escape without the owning view.

`translation_index` is finite only when the cycle-gain subgroup has full rank in the active periodic lattice. `None` means infinite index because the component is lower-dimensional relative to the declared periodic axes.

8 PeriodicNetView

```

@dataclass(frozen=True, slots=True)
class PeriodicNetView:
    source_graph_digest: str
    source_topology_digest: str
    pbc: tuple[bool, bool, bool]
    policy: NetViewPolicy
    vertex_atom_indices: tuple[int, ...]
    edge_keys: tuple[FrameworkEdgeKey, ...]
    vertex_signatures: tuple[NetSignature, ...]
    edge_signatures: tuple[NetSignature, ...]
    components: tuple[PeriodicNetComponent, ...]
    canonical_schema_version: str = CANONICAL_PERIODIC_NET_VIEW_SCHEMA
    digest_algorithm: str = PERIODIC_NET_VIEW_DIGEST_ALGORITHM
    digest: str = ""

```

8.1 Meaning

- `source_graph_digest` binds primitive-ring-compatible graph identity;
- `source_topology_digest` binds the complete source mapping/provenance identity;
- source vertex and edge tuples are exact, ordered mappings to `FrameworkTopology`;
- signatures define which source decorations future automorphisms must preserve;
- components describe quotient connectivity and periodic translation subgroups;
- `digest` binds the exact source plus semantic policy/signatures.

Primitive-ring identity remains

(source graph digest, `PrimitiveRingKey`).

The net-view digest is additional symmetry/tiling provenance and does not replace the source graph digest.

8.2 Convenience properties

```

view.n_vertices: int
view.n_edges: int
view.n_components: int
view.ambient_periodic_rank: int
view.translation_rank: int | None
view.translation_index: int | None
view.lifted_component_count: int | None
view.is_lift_connected: bool
view.natural_tiling_eligible: bool

```

`translation_rank` and `translation_index` are defined directly only for a single quotient component. Per-component values remain available through `view.components` for disconnected views.

8.3 Source lookup

```
view.vertex_position(atom_index: int) -> int
view.edge_position(edge_key: FrameworkEdgeKey) -> int
view.vertex_signature(atom_index: int) -> NetSignature
view.edge_signature(edge_key: FrameworkEdgeKey) -> NetSignature
```

Absent source records raise `PeriodicNetViewInputError`.

8.4 Serialization

```
view.to_dict() -> dict[str, Any]
PeriodicNetView.from_dict(payload, *, topology: FrameworkTopology) -> PeriodicNetView
```

Deserialization requires the exact source topology. The implementation rebuilds the view from the serialized policy and source topology, then verifies source digests, view digest, and canonical payload equality.

9 Builder

```
def build_periodic_net_view(
    topology: FrameworkTopology,
    *,
    policy: NetViewPolicy | None = None,
) -> PeriodicNetView:
    ...
```

If `policy` is omitted, the unlabeled framework-net policy is used.

9.1 Input constraints

- `topology` must be a validated immutable `FrameworkTopology`;
- the first backend accepts no vertex/edge filtering or contraction request;
- the policy must be a `NetViewPolicy`;
- source ordering and multiplicity are preserved exactly;
- nonperiodic axes may not carry nonzero source edge shifts, as already enforced by `FrameworkTopology`.

9.2 Output guarantees

- exact source vertex and edge orbit sets;
- deterministic signatures;
- deterministic component and cycle-gain ordering;
- exact integer rank/index diagnostics;
- stable schema-versioned digest;
- no mutation of the source topology.

10 Periodic translation analysis

10.1 Quotient components

Connected components are inherited from the exact source quotient graph because the first net-view backend does not alter incidence.

For each quotient component, choose a deterministic spanning tree and assign an integer potential $\mathbf{p}_v$ to every vertex. If tree traversal reaches j from i across oriented edge shift Δ , then

$$\mathbf{p}_j = \mathbf{p}_i + \Delta.$$

10.2 Fundamental cycle gains

For every canonical quotient edge

$$e = (i, j, \Delta_e),$$

the corresponding closed-walk gain is

$$\mathbf{g}_e = \mathbf{p}_i + \Delta_e - \mathbf{p}_j.$$

Tree edges produce zero. Nonzero gains from cotree and self-image edges generate the full translation subgroup of closed walks in that quotient component. Generators are sign-normalized, deduplicated, and sorted deterministically.

10.3 Rank

The translation rank is the exact rational rank of the gain vectors projected onto active periodic axes. Because the ambient dimension is at most three, the implementation evaluates integer minors directly rather than using floating-point linear algebra.

10.4 Finite index and lifted connectedness

When the subgroup has full rank d in the active periodic lattice $\mathbb{Z}^d$, its finite index is

$$[\mathbb{Z}^d : \Lambda_G] = \gcd\{|\det M| : M \text{ is a } d \times d \text{ generator minor}\}.$$

This is the maximal determinant divisor from Smith normal form.

The index matters because a connected quotient graph can still generate multiple disconnected lifted nets. For example, gains

$$(2, 0, 0), \quad (0, 1, 0), \quad (0, 0, 1)$$

have rank three but index two. The quotient has one component, while the infinite lift has two translation cosets.

For one quotient component:

`translation_index == 1 <=> connected lift in the declared periodic lattice`

For several full-rank quotient components, `lifted_component_count` is the sum of component indices. It is `None` if any component has lower rank and therefore infinite lattice index.

11 Natural-tiling precondition

The first three-periodic natural-tiling backend accepts a view only when

$$\text{pbc} = (1, 1, 1),$$

$$N_{\text{quotient components}} = 1,$$

$$\text{rank } \Lambda_G = 3,$$

and

$$[\mathbb{Z}^3 : \Lambda_G] = 1.$$

The last condition strengthens the earlier rank-only architecture statement and prevents a quotient-connected but lift-disconnected graph from being treated as one three-periodic net.

12 Digest and equality semantics

The policy digest excludes the descriptive label. The view digest includes:

- schema version;
- source graph digest;
- source topology digest;
- semantic policy digest;
- canonical vertex signatures; and
- canonical edge signatures.

Two views built from the same source and same retained fields compare equal even if their policy labels differ.

The digest is source-bound, not an unlabeled graph-isomorphism invariant. Comparing views created from independently indexed cells or supercells requires a future explicit periodic-net mapping.

13 Edge cases and warnings

13.1 Ignoring a field does not merge records

If two parallel edges receive the same signature, they remain two separate edge orbits. Later automorphisms may exchange them only through an explicit edge permutation.

13.2 Structural filtering is out of scope

Removing vertices, removing edges, or contracting a different graph would change primitive-ring semantics. Such a graph must become a new topology source with a compatible ring catalog.

13.3 Full rank does not imply connected lift

Always inspect `translation_index`; rank alone is insufficient.

13.4 Supercell and hidden-translation interpretation

The view diagnoses the graph relative to the declared source lattice. It does not yet reduce the cell to a minimal translation lattice or discover hidden subcell automorphisms. Those belong to later periodic symmetry discovery.

13.5 Disconnected and lower-dimensional structures

They are valid `PeriodicNetView` results and carry diagnostics, but are rejected by the first three-periodic tiling backend.

13.6 Policy extension

Arbitrary Python signature callbacks are intentionally unsupported because they would not provide deterministic serialization or portable digest semantics. New scientific fields should be added as explicit enum values with tests and schema review.

14 Focused test requirements

The implementation gate must cover:

1. unlabeled versus chemically decorated vertex signatures;
2. semantic policy digest independence from descriptive labels;
3. parallel edge preservation under equal signatures;
4. edge-decoration discrimination under the decorated policy;
5. canonical field ordering and policy serialization;
6. rank-three, index-one connected periodic net;
7. rank-three, index-two quotient-connected but lift-disconnected net;
8. one- and two-periodic diagnostics;
9. zero-periodic finite graph diagnostics;
10. multiple quotient components;
11. source-bound view serialization and mismatch rejection;
12. source lookup failures; and
13. Na-LTA as one rank-three, index-one unlabeled T -net.

15 Non-goals

This stage does not implement:

- automatic periodic-net automorphism discovery;
- adaptation of `ValidatedPeriodicAutomorphism` to net-view ownership;
- minimal-lattice reduction;
- barycentric or crystallographic embeddings;
- strong-ring domain enumeration;
- face candidates, linking, cell complexes, or natural tiling.

16 References

- [1] S. J. Chung, Th. Hahn, and W. E. Klee, “Nomenclature and generation of three-periodic nets: the vector method,” *Acta Crystallographica A* **40**, 42–50 (1984). DOI: 10.1107/S0108767384000088.
- [2] W. E. Klee, “Crystallographic nets and their quotient graphs,” *Crystal Research and Technology* **39**, 959–968 (2004). DOI: 10.1002/crat.200410281.
- [3] O. Delgado-Friedrichs and M. O’Keeffe, “Identification of and symmetry computation for crystal nets,” *Acta Crystallographica A* **59**, 351–360 (2003). DOI: 10.1107/S0108767303012017.
- [4] M. Newman, *Integral Matrices*, Academic Press, New York (1972), especially the determinant-divisor and Smith-normal-form treatment of integer sublattices.